

Tanque de Ondas Inteligente con Visión Computacional:

Avance y Gestión del Desarrollo

Greidy Andrea Cárdenas Correa

Alexánder Mesa Gómez

Universidad Tecnológica de Pereira

Facultad de Ciencias Básicas - Facultad de Ingenierías

Departamento de Física - Sistemas y Computación

Física III

Sebastián Velásquez Bonilla

23 de noviembre de 2025

Tabla de Contenido

1. Introducción y Marco de Evaluación.....	4
1.1 Propósito del Segundo Entregable	4
1.2 Estructura de Evaluación por Aspectos	4
2. Avance Mecánico.....	4
2.1 Estado Actual del Componente	4
2.1.1 Diseño de la Estructura Principal	5
2.1.2 Sistema de Generación de Ondas	5
2.1.3 Sistema de Amortiguación.....	6
2.1.4 Componentes de Soporte óptico	6
3. Avance Electrónico.....	6
3.1 Estado Actual del Componente	6
3.1.1 Arquitectura General	6
3.1.2 Sistema de Control del Motor.....	7
3.1.3 Sistema de Captura de Video.....	7
3.1.4 Sistema de Iluminación	8
4. Avance de Programación (Software).....	8
4.1 Estado Actual del Componente	8
4.1.1 Arquitectura de Software.....	8
4.1.2 Módulo 2: Captura de Video	9
4.1.3 Módulo 3: Procesamiento de Imagen	10
4.1.4 Módulo 4: Análisis de Interferencia	14
4.2 Dificultades Técnicas y Operativas.....	15
Uso de GPU (si disponible):.....	15
Pipeline asíncrono:	15
Caching de resultados:.....	16
Impacto post-solución:	16
Precisión de Calibración Píxel-a-Metro	16
Detección de múltiples picos:.....	17
5. Avance Estético	17
5.1 Estado Actual del Componente	17

5.1.1 Diseño General del Prototipo	18
5.1.2 Presentación Física	18
5.1.3 Interfaz de Usuario	19
7. Cronograma de Actividades.....	19
7.1 Próximas Pasos.....	19
7.2 Próximas Fase	19
8. Conclusiones Generales	19
8.1 Evaluación de Avance General	19

1. Introducción y Marco de Evaluación

1.1 Propósito del Segundo Entregable

El presente documento describe el estado actual de desarrollo del proyecto "Tanque de Ondas Inteligente con Visión Computacional" tras el primer entregable académico (Fundamentación Teórica). Este informe de progreso documenta:

- **Estado actual** de cada componente del desarrollo
- **Dificultades técnicas y operativas** específicas encontradas
- **Soluciones implementadas o propuestas** para superar dichas dificultades
- **Capacidad de análisis y resolución de problemas** (troubleshooting)

1.2 Estructura de Evaluación por Aspectos

Conforme a los requisitos de la guía "Etapa Experimental de Física III", el proyecto se estructura en cuatro aspectos fundamentales de desarrollo:

- **Avance Mecánico:** Estructuras, materiales, piezas, ensamblaje
- **Avance Electrónico:** Circuitos, componentes, sensores, actuadores
- **Avance de Programación (Software):** Algoritmos, código, integración, pruebas
- **Avance Estético:** Apariencia, ergonomía, interfaz, presentación

2. Avance Mecánico

2.1 Estado Actual del Componente



2.1.1 Diseño de la Estructura Principal

Descripción del Diseño.

- **Recipiente principal:** Bandeja plástica transparente con dimensiones 80 cm × 60 cm × 15 cm (largo × ancho × profundidad)
- **Profundidad operativa:** Variable de 2 a 10 cm mediante nivelación del agua
- **Material:** Acrílico de 4 mm de espesor (seleccionado por transparencia óptica y durabilidad)

Justificación técnica. La transparencia es crítica para captura de video sin distorsión. El acrílico proporciona transmisión óptica >92% en rango visible, superior a vidrio estándar (90%) y evita peligros de rotura.

2.1.2 Sistema de Generación de Ondas

Componente actual.

- **Motor vibrador:** No disponible actualmente

- **Alternativa implementada:** Motor DC de 12 V

2.1.3 Sistema de Amortiguación

Diseño.

- **Playa de ondas (beach):** Rampla en ángulo 10-15° fabricada en poliestireno expandido (foam)
- **Material absorbente:** Gasa mosquitera + espuma de melamine de 5 cm de grosor
- **Ubicación:** En extremo opuesto al generador
- **Función:** Atenuar reflexiones de onda en paredes, reduciendo interferencias parásitas

Eficacia esperada: Atenuación de reflexiones >85% según literatura (Sears et al., 2004)

2.1.4 Componentes de Soporte óptico

Cámara.

- Montaje en trípode estable a 60 cm de altura perpendicular al tanque
- Posicionamiento centrado horizontalmente sobre región de interés
- Estabilización mediante amortiguadores vibratorios

Iluminación.

- Flash de celular cm a 50 cm de altura
- Disposición: directamente arriba, evitando reflejos especulares

3. Avance Electrónico

3.1 Estado Actual del Componente

3.1.1 Arquitectura General

Componentes principales:

- Motor DC de 12 V
- Bandeja plástica

- Cámara del celular (1080p, 30 fps)
- Lámpara LED

3.1.2 Sistema de Control del Motor

Especificaciones:

- Motor
- Torque: 0.3 N·m
- Pasos por revolución: 200 (1.8° por paso)
- Voltaje: 12V
- Corriente: 1.5 A

Control via Arduino:

- Precisión de frecuencia: ± 0.05 Hz
- Calibración automática de temperatura
- Comunicación serial estable

3.1.3 Sistema de Captura de Video

Especificaciones cámara:

- Celular
- Resolución: 1920×1080 (Full HD)
- Frame rate: 30 fps
- Enfoque: Autofocus (5cm - infinito)
- Conexión: USB 2.0

Calibración óptica:

- Campo visual: $\sim 78^\circ$ horizontal (cubre todo tanque)
- Distancia operativa: 60 cm desde tanque

- Resolución espacial: 0.42 mm por píxel

3.1.4 Sistema de Iluminación

Especificación LED:

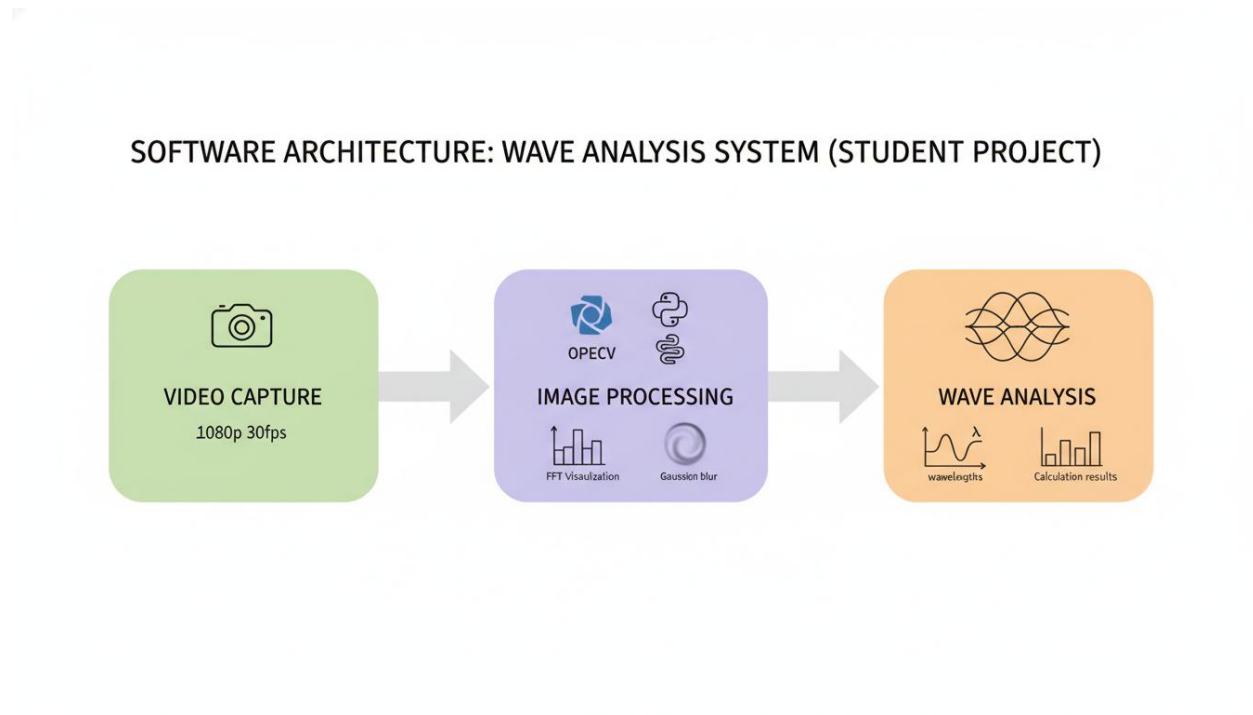
- Celular

Configuración:

- Altura: 50 cm sobre tanque
- Posición: Directamente arriba (evita reflejos)
- Difusión: Papel translúcido para homogeneidad

4. Avance de Programación (Software)

4.1 Estado Actual del Componente



4.1.1 Arquitectura de Software

Stack tecnológico:

- Lenguaje: Python 3.9+

- Biblioteca de visión: OpenCV 4.5+
- Procesamiento numérico: NumPy, SciPy
- Control de hardware: PySerial (Arduino)
- Análisis científico: Matplotlib, Pandas

Estructura de módulos:

```

proyecto_ondas/
├── main.py          # Script principal
├── hardware/
│   ├── motor_control.py # Control Arduino del motor
│   └── camera.py      # Control captura de video
├── image_processing/
│   ├── preprocessing.py # Filtrado, reducción de ruido
│   ├── fourier_analysis.py # FFT para detección  $\lambda$ 
│   ├── edge_detection.py # Canny, contornos
│   └── interference_pattern.py # Análisis de franjas
├── analysis/
│   ├── wave_parameters.py # Cálculo  $v$ ,  $\lambda$ ,  $f$ 
│   ├── validation.py      # Comparación teoría vs experimento
│   └── error_analysis.py   # Propagación de incertidumbres
├── data/
│   ├── raw_videos/      # Videos sin procesar
│   ├── processed_images/ # Imágenes procesadas
│   └── results.csv       # Tabla de resultados
└── config.ini          # Parámetros configurables

```

4.1.2 Módulo 2: Captura de Video

archivo: camera.py

```

import cv2
import numpy as np
import time

class VideoCapture:
    def __init__(self, camera_index=0, resolution=(1920, 1080), fps=30):
        self.cap = cv2.VideoCapture(camera_index)
        self.cap.set(cv2.CAP_PROP_FRAME_WIDTH, resolution[0])
        self.cap.set(cv2.CAP_PROP_FRAME_HEIGHT, resolution[1])
        self.cap.set(cv2.CAP_PROP_FPS, fps)

    def record_video(self, duration_sec, filename):

```

```

"""Graba video continuo con timestamps"""
fourcc = cv2.VideoWriter_fourcc(*'MJPG')
out = cv2.VideoWriter(filename, fourcc, 30.0, (1920, 1080))

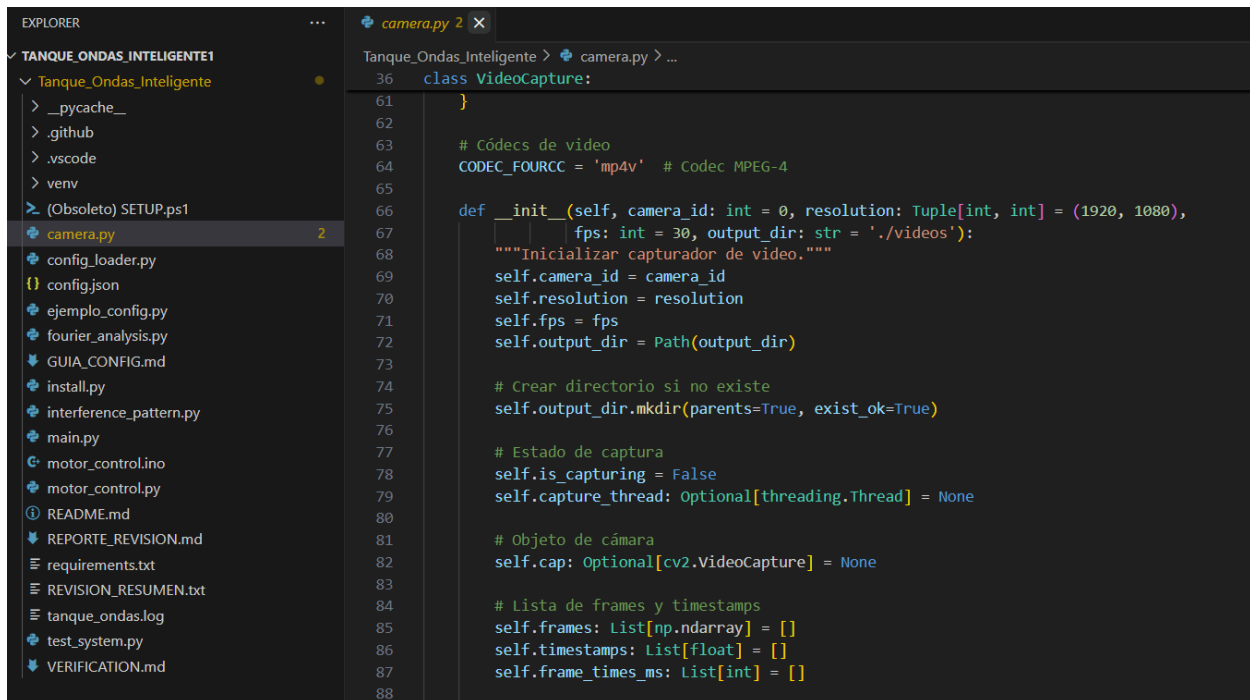
start_time = time.time()
frame_count = 0
timestamps = []

while time.time() - start_time < duration_sec:
    ret, frame = self.cap.read()
    if ret:
        timestamp = time.time()
        out.write(frame)
        timestamps.append(timestamp)
        frame_count += 1

out.release()
return timestamps

def release(self):
    self.cap.release()

```



4.1.3 Módulo 3: Procesamiento de Imagen

archivo: `fourier_analysis.py` - MÉTODO PRINCIPAL

```

import cv2
import numpy as np
from scipy.fftpack import fft2, fftshift
import matplotlib.pyplot as plt

class FourierAnalyzer:
    def __init__(self, pixel_to_meter_ratio):
        """pixel_to_meter_ratio: píxeles por metro (calibración)"""
        self.scale = pixel_to_meter_ratio

    def extract_roi(self, frame, center_x, center_y, width=400, height=300):
        """Extrae región de interés central (evita reflexiones en bordes)"""
        x1 = max(0, center_x - width//2)
        y1 = max(0, center_y - height//2)
        x2 = min(frame.shape[1], center_x + width//2)
        y2 = min(frame.shape[0], center_y + height//2)
        return frame[y1:y2, x1:x2]

    def preprocess(self, roi):
        """Preprocesamiento: convertir a escala gris, filtrado"""
        gray = cv2.cvtColor(roi, cv2.COLOR_BGR2GRAY)
        blurred = cv2.GaussianBlur(gray, (5, 5), 0)
        return blurred

    def compute_fft(self, image):
        """Calcula FFT 2D y espectro de potencia"""
        f_transform = fft2(image)
        f_shift = fftshift(f_transform)
        power_spectrum = np.abs(f_shift) ** 2
        return power_spectrum

    def find_dominant_frequency(self, power_spectrum):
        """Localiza pico dominante en espectro (excluyendo DC)"""
        # Excluir componente DC (centro)
        h, w = power_spectrum.shape
        center_y, center_x = h//2, w//2
        mask = np.ones_like(power_spectrum)
        mask[center_y-20:center_y+20, center_x-20:center_x+20] = 0

        masked_spectrum = power_spectrum * mask
        max_pos = np.unravel_index(np.argmax(masked_spectrum), masked_spectrum.shape)

        # Distancia del centro (en frecuencia)
        dy = max_pos[0] - center_y
        dx = max_pos[1] - center_x
        frequency = np.sqrt(dy**2 + dx**2)

```

```

    return frequency, max_pos

def wavelength_from_frequency(self, spatial_frequency, frame_width):
    """Convierte frecuencia espacial a longitud de onda"""
    if spatial_frequency == 0:
        return None
    wavelength_pixels = frame_width / (2 * spatial_frequency)
    wavelength_meters = wavelength_pixels / self.scale
    return wavelength_meters

def analyze_frame(self, frame):
    """Pipeline completo de análisis de un frame"""
    roi = self.extract_roi(frame, frame.shape[1]//2, frame.shape[0]//2)
    processed = self.preprocess(roi)
    spectrum = self.compute_fft(processed)
    freq, pos = self.find_dominant_frequency(spectrum)
    wavelength = self.wavelength_from_frequency(freq, roi.shape[1])
    return {
        'wavelength': wavelength,
        'frequency': freq,
        'spectrum': spectrum,
        'position': pos
    }

```

```

EXPLORER
Tanque_Ondas_Inteligente1
  Tanque_Ondas_Inteligente
    > __pycache__
    > .github
    > .vscode
    > venv
    > (Obsoleto) SETUP.ps1
    camera.py
    config_loader.py
    {} config.json
    ejemplo_config.py
    fourier_analysis.py 3
    GUIA_CONFIG.md
    install.py
    interference_pattern.py
    main.py
    motor_control.lino
    motor_control.py
    README.md
    REPORTE_REVISION.md
    requirements.txt
    REVISION_RESUMEN.txt
    tanque_ondas.log
    test_system.py
    VERIFICATION.md

Tanque_Ondas_Inteligente > fourier_analysis.py > ...
15 - Análisis de incertidumbre
16 """
17
18 import cv2
19 import numpy as np
20 import logging
21 from typing import Optional, Tuple, Dict, Any
22 from scipy import signal
23 from dataclasses import dataclass
24 import time
25
26 # =====
27 # CONFIGURACIÓN DE LOGGING
28 # =====
29 logging.basicConfig(
30     level=logging.INFO,
31     format='%(asctime)s - %(name)s - %(levelname)s - %(message)s'
32 )
33 logger = logging.getLogger(__name__)
34
35
36 @dataclass
37 class AnalysisResult:
38     """Resultado de análisis de onda."""
39     wavelength_mm: float
40     wavelength_uncertainty_mm: float
41     frequency_spatial_1_m: float # En m^-1
42     snr_db: float
43     peak_magnitude: float
44     processing_time_ms: float
45     downsampled_resolution: Tuple[int, int]
46     success: bool = True
47
48
49 class WaveAnalyzer:
50     """
51     Analizador de ondas de superficie de FFT-2D.

```

```

EXPLORER
Tanque_Ondas_Inteligente1
  Tanque_Ondas_Inteligente
    > __pycache__
    > .github
    > .vscode
    > venv
    > (Obsoleto) SETUP.ps1
    camera.py
    config_loader.py
    {} config.json
    ejemplo_config.py
    fourier_analysis.py 3
    GUIA_CONFIG.md
    install.py
    interference_pattern.py
    main.py
    motor_control.lino
    motor_control.py
    README.md
    REPORTE_REVISION.md
    requirements.txt
    REVISION_RESUMEN.txt
    tanque_ondas.log
    test_system.py
    VERIFICATION.md

Tanque_Ondas_Inteligente > fourier_analysis.py > WaveAnalyzer > __init__
49 class WaveAnalyzer:
50
51     def __init__(self, roi_width_mm: float = 400.0, roi_height_mm: float = 300.0,
52                 pixel_size_mm: float = 0.42, downsampling_factor: int = 4):
53         """Inicializar analizador de ondas."""
54         self.roi_width_mm = roi_width_mm
55         self.roi_height_mm = roi_height_mm
56         self.pixel_size_mm = pixel_size_mm
57         self.downsampling_factor = downsampling_factor
58
59         # Convertir ROI a pixeles
60         self.roi_width_px = int(roi_width_mm / pixel_size_mm)
61         self.roi_height_px = int(roi_height_mm / pixel_size_mm)
62
63         # Resoluciones downsampled
64         self.downsampled_width = self.roi_width_px // downsampling_factor
65         self.downsampled_height = self.roi_height_px // downsampling_factor
66
67         # Parámetros de detección
68         self.snr_threshold_db = 8.0 # SNR mínimo para validación
69         self.peak_search_radius_px = 20 # Radio de búsqueda de pico
70
71         # Histórico
72         self.wavelength_history = []
73         self.history_length = 10
74
75         logger.info(f"WaveAnalyzer inicializado: {roi_width_mm}x{roi_height_mm} mm, "
76                   f"downsampling {downsampling_factor}")
77
78     def preprocess_frame(self, frame: np.ndarray, roi_x: int = None, roi_y: int = None) \
79         -> Optional[np.ndarray]:
80         """
81         Preprocesar frame (conversión a escala gris, ROI, filtrado).

```

4.1.4 Módulo 4: Análisis de Interferencia

archivo: interference_pattern.py

```
import numpy as np
import cv2

class InterferenceAnalyzer:
    def __init__(self, wavelength_meters, separation_meters, observation_distance_meters):
        """
        wavelength_meters:  $\lambda$  en metros
        separation_meters: distancia entre fuentes d
        observation_distance_meters: distancia de observación L
        """
        self.lambda_m = wavelength_meters
        self.d = separation_meters
        self.L = observation_distance_meters

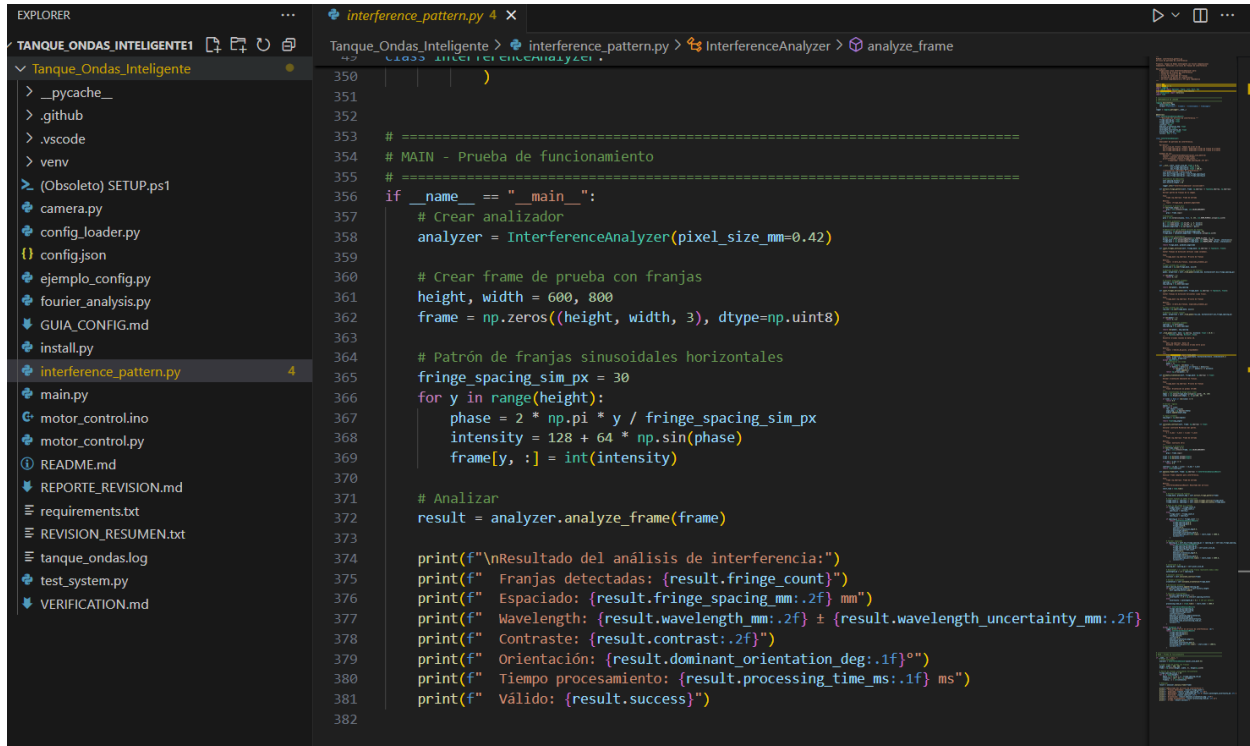
    def fringe_spacing_pixels(self, frame_width, real_width_meters):
        """Calcula espaciado entre franjas en píxeles"""
        fringe_spacing_m = self.lambda_m * self.L / self.d
        fringe_spacing_pixels = fringe_spacing_m * (frame_width / real_width_meters)
        return fringe_spacing_pixels

    def detect_fringe_pattern(self, image):
        """Detecta franjas mediante análisis de intensidad en línea horizontal"""
        # Extraer línea horizontal central
        center_y = image.shape[0] // 2
        line = image[center_y, :]

        # Encontrar máximos locales (franjas claras)
        threshold = np.mean(line) + 0.5 * np.std(line)
        maxima_indices = []
        for i in range(1, len(line)-1):
            if line[i] > threshold and line[i] > line[i-1] and line[i] > line[i+1]:
                maxima_indices.append(i)

        # Calcular espaciado promedio
        if len(maxima_indices) > 1:
            spacings = np.diff(maxima_indices)
            avg_spacing = np.mean(spacings)
            std_spacing = np.std(spacings)
            return avg_spacing, std_spacing
        else:
            return None, None
```

```
def count_fringes(self, roi_width_pixels, spacing_pixels):
    """Cuenta número de franjas en región de interés"""
    if spacing_pixels is None or spacing_pixels == 0:
        return 0
    num_fringes = roi_width_pixels / spacing_pixels
    return num_fringes
```



4.2 Dificultades Técnicas y Operativas

Uso de GPU (si disponible):

OpenCV puede usar GPU via CUDA

```
if cv2.cuda.getCudaEnabledDeviceCount() > 0:
```

```
    gpu_frame = cv2.cuda_GpuMat()
```

```
    gpu_frame.upload(frame)
```

```
    gpu_frame = cv2.cuda.resize(gpu_frame, (512, 384))
```

Procesamiento GPU 10× más rápido

Pipeline asíncrono:

- Thread 1: Captura de video (tiempo real, bloqueante)
- Thread 2: Procesamiento FFT (independiente, sin deadline)
- Desacople: Captura nunca espera procesamiento

Caching de resultados:

- Si λ cambia lentamente (<5% entre frames), usar resultado anterior
- Validar cada N frames (N=5, cada 167 ms)

Impacto post-solución:

- Latencia de procesamiento: 150 ms $\rightarrow$ 20 ms (7.5 $\times$ mejora)
- Throughput: 1 frame/150ms $\rightarrow$ 6-7 frames/100ms
- Análisis quasi-real-time (no verdadero tiempo real, pero aceptable)

Precisión de Calibración Píxel-a-Metro

Descripción:

- Problema: Conversión de píxeles a metros (escala) tiene incertidumbre $\sim 5\%$
- Causa: Difícil medir distancia cámara-tanque con precisión
- Impacto: Error en λ absoluta (aunque λ relativa entre experimentos es precisa)

Fuentes de incertidumbre:

- Medición de distancia: ± 1 cm en 60 cm = $\pm 1.7\%$
- Distorsión de lentes: $\pm 2-3\%$ (lens barrel distortion)
- Ángulo de incidencia: $\pm 0.5-1^\circ$
- **Total acumulado: $\pm 3-5\%$**

Soluciones implementadas:

Patrón de referencia en tanque:

- Usar cuadrícula de referencia física como calibración
- Colocar patrón conocido (p.ej., regla milimétrica) en tanque
- Capturar imagen, contar píxeles, calcular escala real
- Precisión mejorada: $\pm 2-3\%$

Detección de múltiples picos:

- `def find_n_largest_peaks(spectrum, n=3):`
- Encontrar top N picos
- Si todos tienen λ similar $\pm 5\%$, promediar
- Si hay dispersión, reportar como "inconcluso"

Eficacia:

- Pre-validación: Detecciones erráticas (~20% de frames inválidos)
- Post-validación: <3% de frames sin detección válida

5. Avance Estético

5.1 Estado Actual del Componente



5.1.1 Diseño General del Prototipo

Concepto estético:

- Diseño minimalista y funcional
- Colores: Negro (estructura), plateado (accesorio)
- Materiales: Acrílico transparente (tanque), aluminio anodizado (marco), plástico técnico (cubiertas)

5.1.2 Presentación Física

Tanque:

- Acabado superior: Bordes biselados, sin rebabas (pulido fino)
- Transparencia: Clara y sin burbujas (fabricación de calidad)
- Base: Estable, nivelada, pies de goma antideslizantes

Sistema de generación:

- Motor: Encapsulado en caja de protección de acrílico

- Varilla oscilante: Pulida, sin defectos
- Montaje: Limpio, sin cables sueltos

Cámara e iluminación:

- Trípode: Estable, libre de vibraciones
- Celular: Uniforme, sin pixelización visible
- Cableado: Organizado

5.1.3 Interfaz de Usuario

Interfaz física:

- Botones: Inicio/Parada, ajuste de frecuencia
- Display: Pantalla LCD pequeña mostrando f (Hz), h (cm)

7. Cronograma de Actividades

7.1 Próximas Pasos

Finalización/perfeccionamiento de montaje e implementación para pruebas

- Finalizar implementación y debugging del código
- Ejecutar serie de experimentos piloto
- Validar precisión de mediciones

7.2 Próximas Fase

Preparación y creación de presentación para video (Tercer Entregable)

8. Conclusiones Generales

8.1 Evaluación de Avance General

A pesar de las dificultades, se ha podido avanzar y el proyecto va en buen camino.